

DATABASE SYSTEMS

DB Lecture 4

Transactions, Concurrency Control, and Indexing

Topics covered in this lecture:

- Transactions and the ACID properties
- Transaction states and the transaction log
- Concurrency problems: lost update, dirty read, unrepeatable read
- Schedules, serializability, and locking protocols
- Deadlocks: detection and prevention
- Indexing: B+ trees and hash indexes
- Query processing and optimization basics
- A brief introduction to NoSQL and distributed databases

Database Systems Lecture Series
Keeping Databases Correct and Fast Under Concurrent Load

1. What is a Transaction?

A **transaction** is a logical unit of work made up of one or more database operations (reads and writes) that must be executed as an indivisible whole. The classic example is a bank transfer: money must be deducted from one account and added to another, and it must never be the case that only one of these two operations succeeds.

```
BEGIN TRANSACTION;  
UPDATE Account SET Balance = Balance - 500 WHERE AccountID = 'A1';  
UPDATE Account SET Balance = Balance + 500 WHERE AccountID = 'A2';  
COMMIT;
```

If a failure occurs after the first UPDATE but before the second, the transaction must be able to be rolled back so that neither update takes effect, leaving the database exactly as it was before the transaction began.

2. The ACID Properties

A DBMS guarantees that every transaction satisfies four properties, commonly remembered by the acronym **ACID**.

2.1 Atomicity

A transaction is treated as a single, indivisible unit: either all of its operations are applied to the database, or none of them are. If any part of the transaction fails, the entire transaction is rolled back.

2.2 Consistency

A transaction takes the database from one valid state to another valid state, preserving all defined integrity constraints (primary keys, foreign keys, check constraints, and any application-level business rules the transaction is responsible for maintaining).

2.3 Isolation

Concurrently executing transactions must not interfere with one another; the effect of running transactions concurrently must be equivalent to running them one after another in some order (this idea is formalized as **serializability**, covered in Section 4).

2.4 Durability

Once a transaction commits, its changes are permanent and survive any subsequent failure, including a power loss or system crash. This is typically achieved by writing changes to a non-volatile transaction log before or as part of the commit.

3. Transaction States and the Log

3.1 Transaction State Diagram

A transaction moves through a well-defined sequence of states during its lifetime:

- **Active:** the initial state; the transaction is executing its operations.
- **Partially Committed:** the transaction has executed its final operation but has not yet been made permanent.
- **Committed:** the transaction has completed successfully and its effects are permanent.
- **Failed:** the transaction cannot proceed further due to a hardware or logical error.
- **Aborted:** the transaction has been rolled back and the database restored to the state before the transaction began.
- **Terminated:** the transaction has either committed or fully aborted and left the system.

3.2 The Transaction (Write-Ahead) Log

The DBMS maintains a **log** — a sequential, append-only record of every change made to the database, including enough information (typically the old and new values of each modified item) to undo or redo the change. The **write-ahead logging (WAL)** protocol requires that a log record describing a change be written to stable storage before the corresponding data change itself is written to disk. This guarantees that, after a crash, the DBMS's recovery manager can always reconstruct exactly which committed transactions' effects need to be redone, and which uncommitted transactions' effects need to be undone.

3.3 Savepoints

A **SAVEPOINT** marks an intermediate point within a transaction to which the transaction can later be rolled back, without aborting the entire transaction — useful for handling recoverable errors within a larger unit of work.

4. Concurrency Problems

When multiple transactions execute concurrently without coordination, several classic anomalies can occur.

4.1 Lost Update

Two transactions read the same item, then both write back an updated value based on what they read; the second write overwrites the first, and the first transaction's update is silently lost.

```
T1: Read(Balance) -- Balance = 100
T2: Read(Balance) -- Balance = 100
T1: Balance = Balance + 50; Write(Balance) -- writes 150
T2: Balance = Balance - 20; Write(Balance) -- writes 80 (T1's update is lost!)
```

4.2 Dirty Read (Uncommitted Dependency)

A transaction reads a value written by another transaction that has not yet committed. If the writing transaction later aborts, the reading transaction has used a value that never really existed in the database.

4.3 Unrepeatable (Non-Repeatable) Read

A transaction reads the same item twice and gets two different values, because another transaction updated and committed a change to that item in between the two reads.

4.4 Phantom Read

A transaction re-executes a query that returns a set of rows matching a condition, and finds a different set of rows the second time, because another transaction inserted or deleted rows matching that condition in between.

5. Schedules and Serializability

A **schedule** is a sequence describing the chronological order in which the operations of a set of concurrent transactions are executed. Since the DBMS interleaves operations from different transactions to improve throughput, we need a formal way to decide whether a given interleaving is safe.

5.1 Serial vs. Serializable Schedules

- A **serial schedule** executes transactions one completely after another, with no interleaving. Serial schedules are always correct (since each transaction individually preserves consistency) but offer no concurrency, and hence poor performance.
- A **serializable schedule** is an interleaved schedule whose effect on the database is guaranteed to be equivalent to that of some serial schedule of the same transactions. Serializability is the gold-standard correctness criterion for concurrent execution.

5.2 Isolation Levels

Because true serializability can be costly to enforce, SQL defines several weaker **isolation levels** that trade off strictness for performance:

Isolation Level	Prevents
Read Uncommitted	Nothing (dirty reads possible).
Read Committed	Dirty reads.
Repeatable Read	Dirty reads and unrepeatable reads.
Serializable	Dirty reads, unrepeatable reads, and phantom reads.

6. Concurrency Control Protocols

6.1 Lock-Based Protocols

The most common approach to concurrency control uses **locks** to control access to data items. A **shared (S) lock** allows a transaction to read an item and can be held by multiple transactions simultaneously. An **exclusive (X) lock** allows a transaction to both read and write an item and can be held by only one transaction at a time.

6.2 Two-Phase Locking (2PL)

The **Two-Phase Locking protocol** guarantees serializability by requiring every transaction to acquire all the locks it needs before releasing any lock:

- **Growing phase:** the transaction may acquire locks but may not release any.
- **Shrinking phase:** the transaction may release locks but may not acquire any new ones.

A stricter and more commonly implemented variant, **Strict Two-Phase Locking**, holds all exclusive locks until the transaction commits or aborts, which also prevents dirty reads and simplifies recovery.

6.3 Timestamp Ordering

As an alternative to locking, **timestamp-ordering** protocols assign each transaction a unique timestamp when it begins, and ensure that conflicting operations are executed in timestamp order, aborting and restarting any transaction that attempts to violate this order.

6.4 Optimistic (Validation) Concurrency Control

Optimistic protocols let transactions execute freely, buffering their writes, and only check for conflicts at commit time (the **validation phase**). If a conflict is found, the transaction is aborted and restarted. This approach performs well when conflicts are rare.

7. Deadlocks

A **deadlock** occurs when two or more transactions are each waiting for a lock held by another, so that none of them can ever proceed.

```
T1: Lock-X(A) ... waits for Lock-X(B) -- held by T2
T2: Lock-X(B) ... waits for Lock-X(A) -- held by T1
-- Neither T1 nor T2 can proceed: deadlock
```

7.1 Deadlock Prevention

Prevention protocols ensure the system can never enter a deadlocked state, typically by ordering transactions by timestamp and forcing an older transaction to abort a younger one (or vice versa) rather than wait, using schemes such as **wait-die** and **wound-wait**.

7.2 Deadlock Detection

Alternatively, the DBMS can allow deadlocks to occur and periodically check for them by building a **wait-for graph**, where a directed edge from T_i to T_j means T_i is waiting for a lock held by T_j . A cycle in this graph indicates a deadlock; the DBMS resolves it by choosing a **victim** transaction to abort and roll back.

7.3 Deadlock Avoidance

Timeout-based approaches simply abort any transaction that has been waiting longer than a threshold, which is simple to implement but can abort transactions that were not actually deadlocked.

8. Indexing for Efficient Access

Without an index, answering a query such as "find the student with StudentID = 500" requires scanning every row in the table (a **full table scan**). An **index** is an auxiliary data structure that allows the DBMS to locate rows matching a search condition quickly, without scanning the entire table.

8.1 B+ Tree Indexes

The most widely used index structure in relational databases is the **B+ tree**, a balanced, multi-way search tree in which all actual data pointers are stored in the leaf nodes, and internal nodes contain only routing keys used to navigate to the correct leaf. Leaf nodes are typically linked together in sorted order, which makes B+ trees efficient for both equality lookups (e.g. StudentID = 500) and range queries (e.g. GPA BETWEEN 3.0 AND 3.5), and for supporting ORDER BY without an extra sort step.

8.2 Hash Indexes

A **hash index** applies a hash function to the indexed column's value to compute the location of the matching row(s) directly. Hash indexes offer very fast equality lookups ($O(1)$ on average) but cannot efficiently support range queries or sorting, since hashing destroys the ordering of values.

8.3 Clustered vs. Non-Clustered Indexes

- **Clustered index:** the physical order of rows on disk matches the order of the index key. A table can have at most one clustered index (often built automatically on the primary key).
- **Non-clustered (secondary) index:** a separate structure that stores index keys together with pointers to the actual row locations; a table can have several.

8.4 When to Create an Index

Indexes speed up reads but slow down writes, since every INSERT, UPDATE, or DELETE must also update every affected index. Good candidates for indexing include primary keys, foreign keys, and columns frequently used in WHERE clauses, JOIN conditions, or ORDER BY clauses; columns that are rarely queried, or tables that are written far more often than read, are generally poor candidates.

9. Query Processing and Optimization

When the DBMS receives a SQL query, it does not execute it literally as written. Instead, it goes through several stages to find an efficient execution strategy.

9.1 Stages of Query Processing

- 1 **Parsing and translation:** the SQL query is checked for syntax and validity, then translated into an internal representation, typically based on relational algebra.
- 2 **Optimization:** the query optimizer considers multiple equivalent execution plans (e.g. different join orders, whether to use an index or a full scan) and estimates the cost of each, using statistics about table sizes and data distributions maintained in the system catalog.
- 3 **Execution:** the chosen (lowest estimated cost) execution plan is carried out by the query execution engine, and the results are returned to the user.

9.2 The EXPLAIN Statement

Most SQL database systems provide an EXPLAIN (or EXPLAIN PLAN / EXPLAIN ANALYZE) statement that shows the execution plan the optimizer chose for a query, which is an essential tool for diagnosing slow queries in practice.

```
EXPLAIN SELECT Name FROM Student WHERE DeptID = 'D01';  
-- Shows whether an index scan or a full table scan will be used
```

10. A Brief Introduction to NoSQL and Distributed Databases

As applications grew to serve massive numbers of users with rapidly changing, often semi-structured data, a family of alternative database systems known as **NoSQL** ("Not only SQL") emerged, generally trading some of the strict consistency guarantees of relational systems for horizontal scalability and flexible schemas.

10.1 Major NoSQL Categories

Category	Data Model	Example Systems
Document	Semi-structured documents (e.g. JSON)	MongoDB, CouchDB
Key-Value	Simple key-to-value mapping	Redis, DynamoDB
Wide-Column	Rows with dynamic, sparse columns	Cassandra, HBase
Graph	Nodes and edges with properties	Neo4j, Neptune

10.2 The CAP Theorem

The **CAP theorem** states that a distributed data store can provide at most two of the following three guarantees simultaneously: **Consistency** (every read receives the most recent write), **Availability** (every request receives a non-error response), and **Partition tolerance** (the system continues to operate despite network partitions between nodes). Since partition tolerance is essentially mandatory for any distributed system running over an unreliable network, real systems mostly choose between prioritizing consistency (**CP**) or availability (**AP**) when a partition actually occurs.

10.3 BASE vs. ACID

Many NoSQL systems favor a **BASE** (Basically Available, Soft state, Eventually consistent) model over strict ACID guarantees, accepting that different replicas of the same data may briefly disagree, in exchange for higher availability and throughput at scale.

11. Review Questions

- 1 Define the four ACID properties and explain, for each, what could go wrong if it were not guaranteed.
- 2 Describe the lost update, dirty read, and unrepeatable read anomalies, giving a short example schedule for each.
- 3 State the rule of Two-Phase Locking and explain how it guarantees serializability.
- 4 Explain the difference between deadlock prevention and deadlock detection, and describe one concrete algorithm or protocol for each.
- 5 Compare B+ tree indexes and hash indexes. For a column frequently queried with a BETWEEN clause, which would you choose, and why?
- 6 What is the purpose of a query optimizer, and what kind of information does it use to choose between candidate execution plans?
- 7 State the CAP theorem and explain why a distributed system cannot, in general, sacrifice partition tolerance.
- 8 Compare ACID and BASE. Under what circumstances might an application designer prefer a BASE system over a traditional ACID relational database?

12. Summary

This final lecture in the series examined what happens once a well-designed database (DB Lectures 1–3) is put into active, concurrent use. We defined transactions and the ACID properties that guarantee their correctness, examined the concurrency anomalies that can arise without coordination, and introduced locking and other protocols that guarantee serializable, deadlock-free execution. We then turned to performance: how indexes such as B+ trees and hash indexes accelerate data access, and how the query optimizer chooses an efficient execution plan for a given SQL query. Finally, we took a brief look beyond the relational model at NoSQL systems and the trade-offs described by the CAP theorem.

Key Takeaways

- ACID (Atomicity, Consistency, Isolation, Durability) defines what it means for a transaction to execute correctly.
- Concurrency control protocols such as Two-Phase Locking exist to guarantee serializable execution while still allowing useful concurrency.
- Deadlocks are an unavoidable risk of lock-based concurrency control and must be actively prevented or detected and resolved.
- Indexes trade write overhead for dramatically faster reads, and choosing the right index structure depends on the query patterns a table needs to support.
- NoSQL systems trade some ACID guarantees for horizontal scalability, guided by the trade-offs formalized in the CAP theorem.

Series Conclusion

Across these four lectures, we progressed from the fundamental motivation for databases (DB Lecture 1), through the relational model and SQL (DB Lecture 2), sound schema design via normalization (DB Lecture 3), and finally the mechanisms that keep a live, concurrently-used database both correct and performant (DB Lecture 4). Together, these form the essential foundation for further study in areas such as distributed databases, data warehousing, and database internals.

13. Glossary of Key Terms

Term	Definition
Transaction	A logical unit of work executed as an indivisible whole.
ACID	Atomicity, Consistency, Isolation, Durability — the four correctness properties of a transaction.
Serializability	The property that a concurrent schedule's effect matches some serial execution of the same tra
Two-Phase Locking	A locking protocol with distinct growing and shrinking phases that guarantees serializability.
Deadlock	A cycle of transactions each waiting on a lock held by another.
B+ tree	A balanced tree index structure supporting equality and range queries.
Query optimizer	The DBMS component that selects an efficient execution plan for a query.
CAP theorem	A distributed system can guarantee at most two of Consistency, Availability, and Partition tolera

This concludes the four-part DB Lecture series. Reviewing the glossaries at the end of each lecture, alongside the review questions, is a good way to prepare for further study or an assessment covering this material.

14. Series Overview and Further Study

For reference, here is a map of the full DB Lecture series and suggested directions for continued study beyond it.

Lecture	Core Topic
DB Lecture 1	Introduction to databases, DBMS architecture, data models, and the ER model.
DB Lecture 2	The relational model, keys, constraints, relational algebra, and SQL.
DB Lecture 3	Functional dependencies, normalization (1NF-BCNF), and ER-to-relational mapping.
DB Lecture 4	Transactions, ACID, concurrency control, indexing, and query processing.

Suggested Next Steps

- Practice writing and executing SQL queries against a real DBMS such as PostgreSQL, MySQL, or SQLite to reinforce the concepts from DB Lecture 2.
- Design and normalize a schema for a domain of your choosing (e.g. a library, a small online store) as an exercise combining DB Lectures 1, 2, and 3.
- Explore database internals further: storage engines, write-ahead logging implementations, and cost-based query optimization.
- Investigate a NoSQL system such as MongoDB or Redis hands-on to see how the trade-offs discussed in Section 10 manifest in a real product.